UVM Session 2

Stimulus Gen

```
package shared_pkg;
typedef enum {SHIFT, ROTATE} mode_e;
typedef enum {RIGHT, LEFT} direction_e;
endpackage
```

```
class shift_reg_agent extends uvm_agent;
  `uvm_component_utils(shift_reg_agent)
   shift_reg_sequencer sqr;
   shift_reg_driver drv;
   shift_reg_config shift_reg_cfg;
   uvm_analysis_port #(shift_reg_seq_item) agt_ap;

  function new (string name = "shift_reg_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    sqr = shift_reg_sequencer::type_id::create("sqr",this);
    drv = shift_reg_driver::type_id::create("drv",this);
    agt_ap = new("agt_ap", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);
  endfunction
endclass
```

```
class shift_reg_config extends uvm_object;
  `uvm_object_utils(shift_reg_config)

  virtual shift_reg_if shift_reg_vif;

  function new(string name = "shift_reg_config");
    super.new(name);
  endfunction
endclass
```

Kareem Waseem

```
class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
  `uvm_component_utils(shift_reg_driver)
  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item stim_seq_item;
  shift_reg_config shift_reg_cfg;

  function new(string name = "shift_reg_driver", uvm_component parent = null);
     super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Driver - Unable to get configuration object")
    end
  endfunction

  function void connect_phase(uvm_phase phase);
   super.connect_phase(phase);
   shift_reg_vif = shift_reg_cfg.shift_reg_vif;
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
      seq_item_port.get_next_item(stim_seq_item);
      shift_reg_vif.direction = stim_seq_item.direction;
      shift_reg_vif.mode = stim_seq_item.mode;
      shift_reg_vif.datain = stim_seq_item.datain;
      shift_reg_vif.serial_in = stim_seq_item.serial_in;
      shift_reg_vif.reset = stim_seq_item.reset;
      @(negedge shift_reg_vif.clk);
      seq_item_port.item_done();
      `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
    end
  endtask
endclass
```

```
class shift_reg_env extends uvm_env;
  `uvm_component_utils(shift_reg_env)

  shift_reg_driver driver;
  shift_reg_sequencer sqr;

  function new(string name = "shift_reg_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    driver = shift_reg_driver::type_id::create("driver",this);
    sqr = shift_reg_sequencer::type_id::create("sqr",this);
  endfunction: build_phase

  function void connect_phase(uvm_phase phase);
    driver.seq_item_port.connect(sqr.seq_item_export);
  endfunction
endclass
```

Kareem Waseem

```
class shift_reg_main_sequence extends uvm_sequence #(shift_reg_seq_item);
  `uvm_object_utils(shift_reg_main_sequence);
  shift_reg_seq_item seq_item;

  function new (string name = "shift_reg_main_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(10000) begin
      seq_item = shift_reg_seq_item::type_id::create("seq_item");
      start_item(seq_item);
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass
```

```
class shift_reg_reset_sequence extends uvm_sequence #(shift_reg_seq_item);
  `uvm_object_utils(shift_reg_reset_sequence);
  shift_reg_seq_item seq_item;

  function new (string name = "shift_reg_reset_sequence");
    super.new(name);
  endfunction

  task body;
    seq_item = shift_reg_seq_item::type_id::create("seq_item");
    start_item(seq_item);
    seq_item.reset = 1;
    seq_item.serial_in = 0;
    seq_item.direction = direction_e'(0);
    seq_item.mode = mode_e'(0);
    seq_item.datain = 0;
    finish_item(seq_item);
  endtask
endclass
```

```
class shift_reg_seq_item extends uvm_sequence_item;
  `uvm_object_utils(shift_reg_seq_item)

  rand bit reset, serial_in;
  rand direction_e direction;
  rand mode_e mode;
  rand bit [5:0] datain;
  logic [5:0] dataout;

  function new(string name = "shift_reg_seq_item");
  super.new(name);
  endfunction

  function string convert2string();
  return $sformatf("%s reset = 0b%b, serial_in = 0b%b, direction = %s,
  endfunction

  function string convert2string_stimulus();
  return $sformatf("reset = 0b%b, serial_in = 0b%b, direction = %s, mo
  endfunction


  constraint rst_con {
  reset dist {1:= 2, 0:= 98};
  }

  endclass
```

Karem Waseem

```
class shift_reg_sequencer extends uvm_sequencer #(shift_reg_seq_item);
  `uvm_component_utils(shift_reg_sequencer);

  function new (string name = "shift_reg_sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction
endclass
```

```
class shift_reg_test extends uvm_test;
  `uvm_component_utils(shift_reg_test)
  shift_reg_env env;
  shift_reg_config shift_reg_cfg;
  virtual shift_reg_if shift_reg_vif;
  shift_reg_main_sequence main_seq;
  shift_reg_reset_sequence reset_seq;

  function new(string name = "shift_reg_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = shift_reg_env::type_id::create("env",this);
    shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg",this);
    main_seq = shift_reg_main_sequence::type_id::create("main_seq",this);
    reset_seq = shift_reg_reset_sequence::type_id::create("reset_seq",this);

    if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","shift_reg_IF", shift_reg_cfg.shift_reg_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift_reg from the uvm_config_db");

    uvm_config_db #(shift_reg_config)::set(this, "*", "CFG", shift_reg_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: shift_reg_test
```

Monitoring

```
class shift_reg_env extends uvm_env;
  `uvm_component_utils(shift_reg_env)

  shift_reg_agent agt;

  function new(string name = "shift_reg_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = shift_reg_agent::type_id::create("agt",this);
  endfunction: build_phase

endclass
```

Kareem Waseem

```
class shift_reg_agent extends uvm_agent;
  `uvm_component_utils(shift_reg_agent)
   shift_reg_sequencer sqr;
   shift_reg_driver drv;
   shift_reg_config shift_reg_cfg;

  function new (string name = "shift_reg_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    sqr = shift_reg_sequencer::type_id::create("sqr",this);
    drv = shift_reg_driver::type_id::create("drv",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);
  endfunction
endclass
```

```
class shift_reg_monitor extends uvm_monitor;
  `uvm_component_utils(shift_reg_monitor)
  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item rsp_seq_item;

 function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction: build_phase

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
      @(negedge shift_reg_vif.clk);
      rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
      rsp_seq_item.mode = mode_e'(shift_reg_vif.mode);
      rsp_seq_item.datain = shift_reg_vif.datain;
      rsp_seq_item.serial_in = shift_reg_vif.serial_in;
      rsp_seq_item.reset = shift_reg_vif.reset;
      rsp_seq_item.dataout = shift_reg_vif.dataout;
      `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
    end
  endtask
endclass
```

KareemVasseem

```
class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
  `uvm_component_utils(shift_reg_driver)

  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item stim_seq_item;

  function new(string name = "shift_reg_driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
      seq_item_port.get_next_item(stim_seq_item);
      shift_reg_vif.direction = stim_seq_item.direction;
      shift_reg_vif.mode = stim_seq_item.mode;
      shift_reg_vif.datain = stim_seq_item.datain;
      shift_reg_vif.serial_in = stim_seq_item.serial_in;
      shift_reg_vif.reset = stim_seq_item.reset;
      @(negedge shift_reg_vif.clk);
      seq_item_port.item_done();
      `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
    end
  endtask

endclass
```

Checkers

```
class shift_reg_scoreboard extends uvm_scoreboard;
`uvm_component_utils(shift_reg_scoreboard)
uvm_analysis_export #(shift_reg_seq_item) sb_export;
uvm_tlm_analysis_fifo #(shift_reg_seq_item) sb_fifo;
shift_reg_seq_item seq_item_sb;
logic [5:0] shift_reg_out_ref;

int error_count = 0;
int correct_count = 0;

 function new(string name = "shift_reg_scoreboard", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  sb_export = new("sb_export", this);
  sb_fifo = new("sb_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
   super.connect_phase(phase);
   sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    sb_fifo.get(seq_item_sb);
    ref_model(seq_item_sb);
    if (seq_item_sb.dataout != shift_reg_out_ref) begin
      `uvm_error("run_phase", $sformatf("Comparsion failed, Transaction received by the DUT:%s  While the reference ou
      error_count++;
    end
    else begin
      `uvm_info("run_phase", $sformatf("Correct shift_reg out: %s ", seq_item_sb.convert2string()), UVM_HIGH);
      correct_count++;
    end
  end
endtask

task ref_model(shift_reg_seq_item seq_item_chk);
  if (seq_item_chk.reset)
    shift_reg_out_ref = 0;
  else
```

Kareem Waseem

```
  else
    case (seq_item_chk.mode)
      SHIFT:
        if (seq_item_chk.direction == LEFT)
          shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.serial_in};
        else if (seq_item_chk.direction == RIGHT)
          shift_reg_out_ref = {seq_item_chk.serial_in, seq_item_chk.datain[5:1]};
      ROTATE:
        if (seq_item_chk.direction == LEFT)
          shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.datain[5]};
        else if (seq_item_chk.direction == RIGHT)
          shift_reg_out_ref = {seq_item_chk.datain[0], seq_item_chk.datain[5:1]};
    endcase
endtask


function void report_phase(uvm_phase phase);
  super.report_phase(phase);
  `uvm_info("report_phase", $sformatf("Total successful transactions: %0d",correct_count), UVM_MEDIUM);
  `uvm_info("report_phase", $sformatf("Total failed transactions: %0d",error_count), UVM_MEDIUM);
endfunction

endclass
```

```
class shift_reg_coverage extends uvm_component;
`uvm_component_utils(shift_reg_coverage)
uvm_analysis_export #(shift_reg_seq_item) cov_export;
uvm_tlm_analysis_fifo #(shift_reg_seq_item) cov_fifo;
shift_reg_seq_item seq_item_cov;

covergroup cvr_grp;
    datain_cvp : coverpoint seq_item_cov.datain;
    serialin_cvp : coverpoint seq_item_cov.serial_in;
    direction_cvp : coverpoint seq_item_cov.direction;
    mode_cvp : coverpoint seq_item_cov.mode;
endgroup

 function new(string name = "shift_reg_coverage", uvm_component parent = null) ;
    super.new(name, parent);
    cvr_grp = new();
 endfunction

function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  cov_export = new("cov_export", this);
  cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    cov_fifo.get(seq_item_cov);
    cvr_grp.sample();
  end
endtask

endclass
```

Karim Wasem

```
class shift_reg_agent extends uvm_agent;
  `uvm_component_utils(shift_reg_agent)
   shift_reg_sequencer sqr;
   shift_reg_driver drv;
   shift_reg_monitor mon;
   shift_reg_config shift_reg_cfg;
   uvm_analysis_port #(shift_reg_seq_item) agt_ap;

  function new (string name = "shift_reg_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    sqr = shift_reg_sequencer::type_id::create("sqr",this);
    drv = shift_reg_driver::type_id::create("drv",this);
    mon = shift_reg_monitor::type_id::create("mon",this);
    agt_ap = new("agt_ap", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);
    mon.mon_ap.connect(agt_ap);
  endfunction
endclass
```

```
class shift_reg_monitor extends uvm_monitor;
  `uvm_component_utils(shift_reg_monitor)
  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item rsp_seq_item;
  uvm_analysis_port #(shift_reg_seq_item) mon_ap;

 function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction: build_phase

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
      @(negedge shift_reg_vif.clk);
      rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
      rsp_seq_item.mode = mode_e'(shift_reg_vif.mode);
      rsp_seq_item.datain = shift_reg_vif.datain;
      rsp_seq_item.serial_in = shift_reg_vif.serial_in;
      rsp_seq_item.reset = shift_reg_vif.reset;
      rsp_seq_item.dataout = shift_reg_vif.dataout;
      mon_ap.write(rsp_seq_item);
      `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
    end
  endtask
endclass
```

Kareem Waseem

```
class shift_reg_env extends uvm_env;
  `uvm_component_utils(shift_reg_env)

  shift_reg_agent agt;
  shift_reg_scoreboard sb;
  shift_reg_coverage cov;

  function new(string name = "shift_reg_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = shift_reg_agent::type_id::create("agt",this);
    sb = shift_reg_scoreboard::type_id::create("sb",this);
    cov = shift_reg_coverage::type_id::create("cov",this);
  endfunction: build_phase

  function void connect_phase(uvm_phase phase);
    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);
  endfunction
endclass
```

Using covert2string example

```
class usb_2_0_seq_item extends uvm_sequence_item;
 `uvm_object_utils(usb_2_0_seq_item)

  rand bit reset, serial_in;
  rand direction_e direction;
  rand mode_e mode;
  rand bit [5:0] datain;
  logic [5:0] dataout;

 function new(string name = "usb_2_0_seq_item");
  super.new(name);
 endfunction

 function string convert2string();
  return $sformatf("%s reset = 0b%b, serial_in = 0b%b, direction = %s, mode = %s, datain = 0b%b, dataout = 0b%b"
                   ,super.convert2string(), reset, serial_in, direction, mode, datain, dataout);
 endfunction
endclass

class usb_2_1_seq_item extends usb_2_0_seq_item;
 `uvm_object_utils(usb_2_1_seq_item)

 rand bit enable;

 function new(string name = "usb_2_1_seq_item");
  super.new(name);
 endfunction

 function string convert2string();
  return $sformatf("%s , enable = 0b%b ", super.convert2string(), enable);
 endfunction
endclass

module temp;
   usb_2_1_seq_item seq_item = new();

   initial begin
    seq_item.convert2string();
   end
endmodule
```